

Homework 3: Teleoperation

Overview and Deliverables

1. Reading & Reflection
 - 1:** A short (~300 word) reflection
2. CITI Training (2 weeks)
 - a. A PDF of your **completion certificate**
3. Teleoperation
 - 3.1:** Your **answers** to the questions
 - 3.4:** Your **answers** to the questions
 - 3.5.6:** A **video** of your safe gamepad teleoperation
 - 3.6.4:** A **video** of your keyboard teleoperation
 - 3.6.5:** Your updated GitHub repository
 - 3.7:** Your **answers** to the questions

PART 1: Reading & Reflection

Read the following:

1. Kaitlynn Taylor Pineda, Ethan Brown, and Chien-Ming Huang. 2025. “ "See You Later, Alligator": Impacts of Robot Small Talk on Task, Rapport, and Interaction Dynamics in Human-Robot Collaboration.” In *Proceedings of the 2025 ACM/IEEE International Conference on Human-Robot Interaction (HRI '25)*. IEEE Press, 819–828.
2. **Optional additional reading:** Jocelyn Shen, Audrey Lee, Sharifa Alghowinem, River Adkins, Cynthia Breazeal, and Hae Won Park. 2025. “Social Robots as Social Proxies for Fostering Connection and Empathy Towards Humanity.” In *Proceedings of the 2025 ACM/IEEE International Conference on Human-Robot Interaction (HRI '25)*. IEEE Press, 989–999.

Write a short reflection (~300 words) using the following prompts as guidance. Note that you may also address other topics such as something you found surprising, connections you made to the real world or media, etc.

1. What are your thoughts on the question posed in Section VII.E. “Do we really want human-robot small talk at work?” What might some of the technical, social, and ethical implications of this behavior?

PART 2: CITI Training

The humans are, of course, a major part of *human-robot* interaction! Thus, it is important to be aware of best practices when developing technology for people, evaluating our systems with people, etc.

1. Following the instructions on the SFSU Human Subjects Research (IRB) [page](#) (under the “Online CITI Research Training” dropdown), register and complete the **Social/Behavioral Research Course** [CITI training](#).
2. Upload your **completion certificate** to Canvas. Note that you will have two weeks to complete this part of the assignment.

PART 3: Teleoperation

We will finally work with the robots this week! Quanser provides a series of [user manuals for their robots](#) on GitHub; I will do my best to extract or direct you to the relevant information.

3.1 Understand the basics of interfacing with the QBots

Your main goal for this assignment will be to write a ROS package that allows you to **teleoperate the QBot by taking input from the gamepad controller or keyboard**.

There are some [example ROS 2 packages and nodes](#) on GitHub. Before starting to code, review their documentation and code to familiarize yourself with some of the methods, topics, and data types used. Identify the following information:

1. Sensors:
 - a. How many sensors does the QBot have?
 - b. What are they?
2. QUARC executable:
 - a. What is the significance of their custom QUARC executable `qbot_platform_driver_physical.rt-linux_qbot_platform`?
 - b. Why must it be run whenever we want to interface with the robot?
3. Twist messages:
 - a. How is the Twist type (from the `geometry_msgs` library) used in [command.cpp](#)?
 - b. What are its attributes (hint: it might be helpful to check [the ROS 2 documentation](#))?
 - c. What do these attributes represent in this context?
 - d. To what topic are these values being published?
4. Receiving movement commands:
 - a. What method is called when [qbot_platform_driver_interface.cpp](#) receives a Twist message from `command.cpp`?
 - b. What is `receive_buffer`, and what data is stored in each unit of this buffer?
 - c. When and from where are the values of `receive_buffer` updated?

- d. What is send_buffer, and what data is stored in each unit of this buffer?
- e. When and to where does send_buffer get sent?
- f. Where is the wheel speed defined?

Submit your answers to Canvas.

3.2 Prepare the hardware

Batteries: The robots use two batteries, which we will remove, charge, and replace between uses. Each robot comes with an Optimate battery charger. See the [Power User Manual](#) for more details.

1. The batteries are located under the “wings” which can be opened by gently pulling either side of the robot’s outer casing outward until the magnet releases.



a. Empty battery bay (right bay #1)



b. battery bay with LFP battery connected to QBot Platform

2. Unplug and remove the batteries from the robot, and plug them in to charge.



e. Fully Drained Battery/Tune



f. Fully Charged

3. Once charged, replace and connect them back to the robot. Please make sure the wires are not blocking the tracks when you close the wings.

Controller: Each QBot also comes with a game controller which we can use to teleoperate the robot. If it is not already, plug in the USB receiver for the controller to the respective robot.

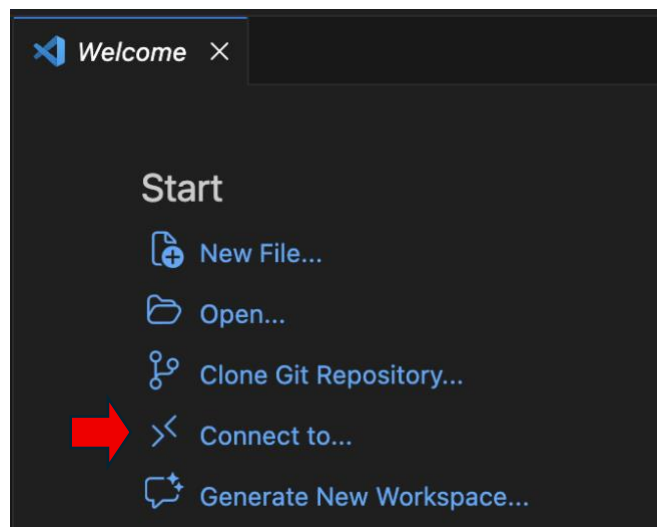
Router: Plug in the internet router if it is not already plugged in.

3.3 Turn on and connect to the robots

1. You can turn on the robot with the round silver power button on the back side of the robot. The LED strips around the bottom of the robot should turn red.
2. Once the robot fully boots up, the LCD display should show the IP address that the robot is connected to. It will automatically connect to the provided router.



3. Connect your local development machine (e.g., laptop) to the wifi router.
 - a. Wifi username: Quanser_UVS-5G
Wifi password: UVS_wifi
4. Now using your preferred development environment (e.g., terminal, VS Code), you can SSH into the robot. Note that for simplicity in this class, we will all use the same user account and password for the robots. Please only modify code in your (or your team's) own cloned GitHub repository.
 - a. QBot username: nvidia
QBot password: nvidia
 - b. e.g., `ssh nvidia@192.168.111.31`



5. Clone your GitHub repository to the robot **within the home directory (/home/nvidia/)** on the robot. As long as you are careful about where you are developing (on the robot itself vs. on your Docker image), you should be able to develop, test, and execute your code on the robot.

3.4 Gain familiarity with the existing teleoperation functionality

Launch the `qbot_platform_manual_drive_launch.py` file and play with operating the robot with the gamepad controller. Answer the following questions (you may find [controller.cpp](#) helpful):

1. What button do you press to “arm” the robot to enable movement?
2. What button do you press to move the robot forward?
3. What button do you press to move the robot in reverse?
4. What button do you press to rotate the robot?

Submit your answers to Canvas.

3.5 Write a ROS package to safely teleoperate the robot via a gamepad

The existing teleoperation nodes offer limited safety supports including a fixed speed, lack of emergency stops, and lack of visual cues. Let’s modify the code to fix that.

1. Create a new ROS package in your `ros2_ws` workspace called *teleop* (see the [ROS 2 tutorial](#) if you need a refresher on how to do this). This will be where you write any code and store relevant files for this assignment.

Note, you may use any nodes you write in future assignments, including having the robot follow a person, move autonomously, and take human input, so think strategically about how to organize your code to save you time on future assignments!

2. Copy the `qbot_platform_manual_drive_launch.py` launch file into your teleop package and update it as follows:
 - a. Change the package name from `qbot_platform` to `teleop` where appropriate (note that for the QUARC executable, you may need to copy it into your teleop package or call the copy in the `qbot_platform` package).
 - b. Remove nodes unrelated to the joystick and driver interface
3. Copy any `.cpp` files for nodes you kept in your launch file into your teleop package. Update these files such that:
 - a. Holding RB while moving the robot makes it move at half of its normal speed.
 - b. The X button sends a “kill” signal to stop the robot and associated program.
 - c. Change the color of the robot’s LED depending on the direction it is moving (e.g., green when moving forward, blue when moving backward, yellow when stationary).
 - d. *Bonus*: Map the controls to a more intuitive layout. Make sure to document the changes you made, if any.
 - e. *Bonus*: Map the robot’s velocity to the magnitude of the joystick (e.g., a “light” push results in a slow movement, while a “hard” push results in a faster movement).
 - f. Remove unnecessary code (such as camera input from the driver interface).
 - g. Add a program header, comments, and docstrings to each file.

4. Build your package, test, and debug your code until it behaves as expected.
5. Record a video demonstrating each of the functionalities you implemented (a, b, c; optionally d and/or e). Video requirements:
 - a. Gamepad input and robot movements are clearly visible for each functionality
 - b. Written or spoken description of how your code works
6. **Upload this video to Canvas.**

3.6 Write a ROS node to teleoperate the robot via the keyboard

Sometimes it is valuable to have multiple ways of interfacing with a system. Let's write a ROS node to control the robot via keyboard input, as an alternative to the gamepad input we already wrote.

1. Write a new ROS node called `keyboard_input` that has the same basic functionality as `controller.cpp`. This can be written in either Python or C++ depending on your preference, and you may choose what specific key mappings you use.
 - a. "Arm" the robot to enable movement
 - b. Move forward and backward
 - c. Rotate left and right
 - d. Move at half speed
 - e. "Kill" signal
 - f. The robot's LEDs change color depending on the direction it is moving
 - g. Add a program header, comments, and docstrings
2. Build your package, test, and debug your code until it behaves as expected.
3. Record a video demonstrating each functionality of your keyboard interface. Requirements:
 - a. Keyboard input and robot movements are clearly visible for each functionality
 - b. Written or spoken description of how your code works
4. **Upload this video to Canvas.**
5. **Commit and push your code to GitHub.**

3.7 A brief reflection

Answer the following questions and **upload them to Canvas**.

1. What other quality of life and/or safety functionalities might you add to your system?
2. How might you leverage the robot's built-in sensors to improve safety of the robot?
3. Why might it be valuable to have multiple input modalities (e.g., gamepad, keyboard)?
4. What are some strengths and limitations of a fully teleoperated robot with regards to control and safety?